

Program – 10

Title:

Demonstrating Kubernetes Dashboard Using CLI and Kubernetes Cluster

Objective:

To deploy an Nginx application on a Kubernetes cluster, expose it through a service, and monitor the cluster resources using the Kubernetes Dashboard.

1. Project Initialization

Before deploying any Kubernetes resources, a dedicated project directory is created to organize all configuration files efficiently.

mkdir new-program-10

Creates a new directory to store the project files.

cd new-program-10

Navigates into the project directory to set the working context.

2. Configuration File Creation

The nano text editor is used to create and edit YAML configuration files that define the desired state of the Kubernetes cluster.

nano deployment.yaml

Creates a Deployment configuration file that specifies the Nginx container image, replica count, and pod labels.

nano services.yaml

Creates a Service configuration file that defines how the Nginx pods are exposed to external traffic using a NodePort service.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash + v [ ] [ ] ... | [ ] [ ] X

• rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ minikube start
🐹 minikube v1.37.0 on Fedora 43
🌟 Using the qemu2 driver based on existing profile
👍 Starting "minikube" primary control-plane node in "minikube" cluster
🔄 Restarting existing qemu2 VM for "minikube" ...
🔧 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
  • Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: default-storageclass, storage-provisioner
🏁 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default

• rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ kubectl apply -f ./deployment.yaml
deployment.apps/nginx-deployment created
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ kubectl apply -f ./service.yaml
service/nginx-service created
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6f9664446b-2599x   1/1     Running   0           31s
nginx-deployment-6f9664446b-lbr2k   1/1     Running   0           31s
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ kubectl get services
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes      ClusterIP   10.96.0.1     <none>       443/TCP          18h
nginx-service   NodePort    10.111.242.116 <none>       80:30465/TCP    29s
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ kubectl get deployments
NAME             READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment 2/2      2             2           64s
```

3. Applying Configurations

The `kubectl apply` command is used to submit YAML configuration files to the Kubernetes API server, which then creates and manages the defined resources.

Command: `kubectl apply -f deployment.yaml`
Purpose: Deploys the Nginx application by creating Pods according to the defined deployment strategy. In this scenario, the deployment successfully created two running Nginx Pods.

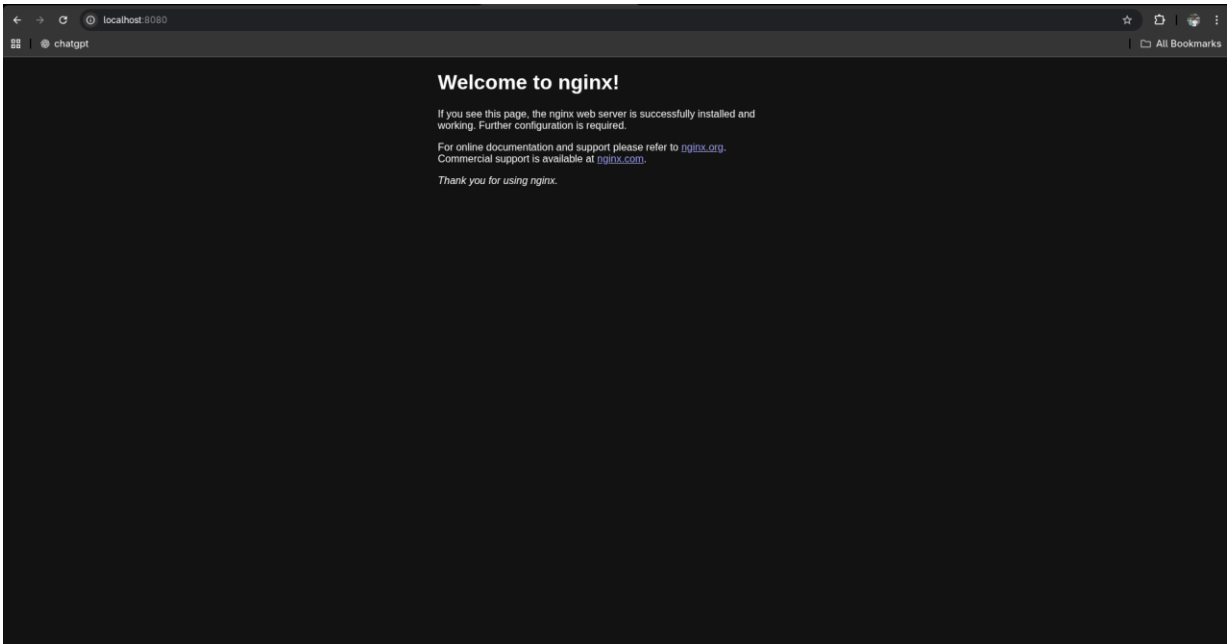
Command: `kubectl apply -f services.yaml`
Purpose: Creates a Kubernetes Service that acts as a logical abstraction to route network traffic to Pods labeled with `app: nginx`.

4. Verification and Status Checks

After applying the configurations, it is important to confirm that all Kubernetes resources are functioning correctly and are in the expected state.

Command	Purpose
<code>kubectl get pods</code>	Lists all active pods. Shows status (e.g., Running), readiness, and age.
<code>kubectl get services</code>	Displays the Service Type (NodePort), Cluster-IP, and the assigned Port (e.g., 32649).
<code>kubectl get deployments</code>	Shows the high-level status of the deployment, confirming if the desired number of replicas are updated and available.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ kubectl get svc
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)        AGE
kubernetes    ClusterIP   10.96.0.1    <none>        443/TCP        18h
nginx-service  NodePort    10.111.242.116 <none>        80:30465/TCP   13m
rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ minikube service nginx-service
```



5. Accessing the Application

Since Minikube runs a Kubernetes cluster locally, the deployed service is not exposed through a public IP address by default.

Command: minikube service nginx-service

Action: Generates a local access URL (for example, <http://192.168.49.2:32649>) and automatically launches the default web browser, displaying the “Welcome to nginx!” page to confirm successful deployment.

6. Cluster Visualization

To visually monitor cluster resources, labels, namespaces, and overall health, Kubernetes provides a web-based dashboard.

Command: minikube dashboard

- Action: Starts a local proxy and opens the Kubernetes Dashboard in the web browser.
- Insight: The dashboard presents a Workload Status overview, clearly showing the deployed resources—1 Deployment, 2 Pods, and 1 ReplicaSet—with green indicators that confirm the cluster is healthy and running as expected.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
rv24mc081_r_mahesh_babu@fedora:~/Downloads/new-program-10$ minikube dashboard
Enabling dashboard ...
  • Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
  • Using image docker.io/kubernetesui/dashboard:v2.7.0
Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

😄 Verifying dashboard health ...
🚀 Launching proxy ...
😄 Verifying proxy health ...
🔗 Opening http://127.0.0.1:36147/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in your default browser...
Opening in existing browser session.
```

